

CSE 401: Numerical Analysis, Simulation and Modeling

Eigenvalue Decomposition: The Power Method

Nafis Tahmid

Department of CSE, BUET

A Matrix Is Not Just a Grid of Numbers

You already know how to multiply a matrix by a vector:

$$\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

But what does this *mean*?

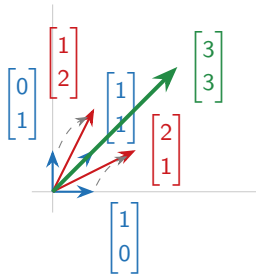
A matrix is a **machine** that eats a vector and spits out another vector.

- You feed it a point in 2D space → it gives you a *different* point.

This “moving points around” is called a **linear transformation**.

Let's Watch a Matrix Work

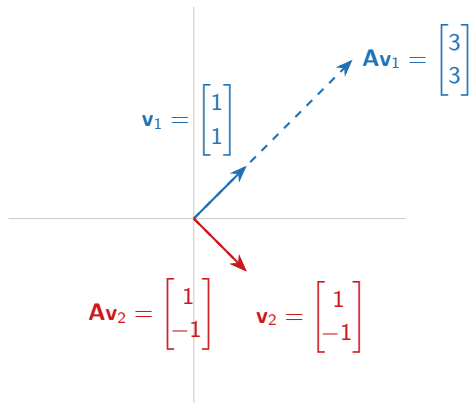
Let $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$. Feed it some vectors and see what comes out.



$\begin{bmatrix} 1 \\ 0 \end{bmatrix}$ & $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$ changed direction, but $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ **stayed on the same line!** It just got **longer**.

That's suspicious. Let's investigate.

Some Vectors Are Stubborn



$$A \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 3 \end{bmatrix} = 3 \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Same direction, $3\times$ longer.

$$A \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \end{bmatrix} = 1 \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

Same direction, same length!

These stubborn vectors that **refuse to change direction** are called...

Eigenvectors

(German: “eigen” = “own” or “characteristic.” The matrix’s *own* special directions.)

Eigenvectors & Eigenvalues

An **eigenvector** $\mathbf{v}$ of matrix $\mathbf{A}$ is a nonzero vector whose **direction does not change** when multiplied by $\mathbf{A}$.

The matrix can only **stretch** (or shrink, or flip) it:

$$\mathbf{A}\mathbf{v} = \lambda \mathbf{v}$$

- $\mathbf{v}$: the **eigenvector** — the “stubborn direction.”

λ : the **eigenvalue** — the “stretch factor.”

In plain English: “When $\mathbf{A}$ acts on $\mathbf{v}$, it doesn’t rotate $\mathbf{v}$ to some random new direction. It just multiplies its length by λ .”

If $\lambda > 1$: stretches. If $0 < \lambda < 1$: shrinks. If $\lambda < 0$: flips direction. If $\lambda = 0$: crushes to zero.

Let's Verify This With Actual Numbers

$$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

Claim 1: $\mathbf{v}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ is an eigenvector with $\lambda_1 = 3$.

Check: $\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \cdot 1 + 1 \cdot 1 \\ 1 \cdot 1 + 2 \cdot 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 3 \end{bmatrix} = 3 \begin{bmatrix} 1 \\ 1 \end{bmatrix} = 3\mathbf{v}_1 \quad \checkmark$

Claim 2: $\mathbf{v}_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$ is an eigenvector with $\lambda_2 = 1$.

Check: $\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 2 \cdot 1 + 1 \cdot (-1) \\ 1 \cdot 1 + 2 \cdot (-1) \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \end{bmatrix} = 1 \begin{bmatrix} 1 \\ -1 \end{bmatrix} = 1\mathbf{v}_2 \quad \checkmark$

So $\mathbf{A}$ has two eigenvectors: $\mathbf{v}_1$ (stretched by 3) and $\mathbf{v}_2$ (stretched by 1, i.e. unchanged).

We want $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$. Rearrange:

$$\mathbf{A}\mathbf{v} - \lambda\mathbf{v} = \mathbf{0} \quad \implies \quad (\mathbf{A} - \lambda\mathbf{I})\mathbf{v} = \mathbf{0}$$

For a nonzero $\mathbf{v}$ to exist:

$$\det(\mathbf{A} - \lambda\mathbf{I}) = 0$$

(If the determinant were nonzero, the only solution would be $\mathbf{v} = \mathbf{0}$. Boring.)

For our 2×2 example:

$$\begin{aligned} \det\left(\begin{bmatrix} 2-\lambda & 1 \\ 1 & 2-\lambda \end{bmatrix}\right) = 0 &\implies (2-\lambda)^2 - 1 = 0 \implies \lambda^2 - 4\lambda + 3 = 0 \\ \implies (\lambda - 3)(\lambda - 1) = 0 \end{aligned}$$

Finding the Eigenvectors

For each λ , plug back into $(\mathbf{A} - \lambda\mathbf{I})\mathbf{v} = \mathbf{0}$ and solve.

$$\begin{aligned} \text{For } \lambda_1 = 3: \quad & \begin{bmatrix} 2-3 & 1 \\ 1 & 2-3 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \Rightarrow \begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \Rightarrow v_1 = v_2 \\ \Rightarrow \quad & \mathbf{v}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \checkmark \end{aligned}$$

$$\begin{aligned} \text{For } \lambda_2 = 1: \quad & \begin{bmatrix} 2-1 & 1 \\ 1 & 2-1 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \Rightarrow \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \Rightarrow v_1 = -v_2 \\ \Rightarrow \quad & \mathbf{v}_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix} \checkmark \end{aligned}$$

No surprises so far. But what can we *do* with these eigenvectors? A lot, it turns out.

Eigenvalue Decomposition —

$$\mathbf{A} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1}$$

A Detour: Coordinate Systems

Normally, we describe points using the standard axes $\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ and $\mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$.

But you can describe the *same point* using *different reference directions*.

Analogy: Giving directions.

- Standard: “Go 3 blocks East and 1 block North.”

Eigenvector: “Go 2 steps along the diagonal, then 1 step along the anti-diagonal.”

Both descriptions reach the **same point** $\begin{bmatrix} 3 \\ 1 \end{bmatrix}$. Just different languages.

Eigenvector Coordinates — By Example

Our eigenvectors: $\mathbf{v}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$, $\mathbf{v}_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$.

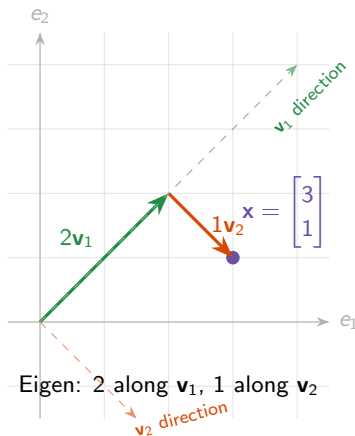
Claim: $\mathbf{x} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$ can be written as $2\mathbf{v}_1 + 1\mathbf{v}_2$.

Check: $2 \begin{bmatrix} 1 \\ 1 \end{bmatrix} + 1 \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \end{bmatrix} + \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 3 \\ 1 \end{bmatrix} \checkmark$

In **standard coordinates**: $\mathbf{x} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$ (3 along $\mathbf{e}_1$, 1 along $\mathbf{e}_2$)

In **eigenvector coordinates**: $\mathbf{x}_{\text{eigen}} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$ (2 along $\mathbf{v}_1$, 1 along $\mathbf{v}_2$)

Visualizing the Two Coordinate Systems



The $\mathbf{V}$ Matrix: Translating Between Languages

Pack the eigenvectors as **columns** of a matrix $\mathbf{V}$:

$$\mathbf{V} = \begin{bmatrix} \mathbf{v}_1 & \mathbf{v}_2 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

What does $\mathbf{V}$ do? It converts **eigenvector coordinates** $\rightarrow$ **standard coordinates**:

$$\mathbf{V} \begin{bmatrix} c_1 \\ c_2 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \end{bmatrix} = c_1 \begin{bmatrix} 1 \\ 1 \end{bmatrix} + c_2 \begin{bmatrix} 1 \\ -1 \end{bmatrix} = c_1 \mathbf{v}_1 + c_2 \mathbf{v}_2$$

Example: $\mathbf{V} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = 2 \begin{bmatrix} 1 \\ 1 \end{bmatrix} + 1 \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$

$\mathbf{V}$ is a **translator**: give it eigen-coordinates, it returns the real-world vector.

$\mathbf{V}^{-1}$: The Reverse Translator

If $\mathbf{V}$ converts eigen $\rightarrow$ standard, then $\mathbf{V}^{-1}$ converts **standard** $\rightarrow$ **eigen**.

$$\mathbf{V}E = X \Rightarrow \mathbf{V}^{-1}X = E$$

For $\mathbf{V} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$, the inverse is: $\mathbf{V}^{-1} = \frac{1}{-2} \begin{bmatrix} -1 & -1 \\ -1 & 1 \end{bmatrix} = \begin{bmatrix} 1/2 & 1/2 \\ 1/2 & -1/2 \end{bmatrix}$

Check: $\mathbf{V}^{-1} \begin{bmatrix} 3 \\ 1 \end{bmatrix} = \begin{bmatrix} 1/2 & 1/2 \\ 1/2 & -1/2 \end{bmatrix} \begin{bmatrix} 3 \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{3}{2} + \frac{1}{2} \\ \frac{3}{2} - \frac{1}{2} \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix} \quad \checkmark$

So $\mathbf{V}^{-1} \begin{bmatrix} 3 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$ means: "In eigenvector language, $\begin{bmatrix} 3 \\ 1 \end{bmatrix}$ is 2 parts $\mathbf{v}_1$ and 1 part $\mathbf{v}_2$."

Wait — We Have $\mathbf{V}^{-1}$ and $\mathbf{V}$. Are We Done?

So far we have two translators:

$\mathbf{V}^{-1}$: converts standard $\rightarrow$ eigen-coordinates

$\mathbf{V}$: converts eigen-coordinates $\rightarrow$ standard

But translating back and forth **by itself does nothing useful**:

$$\mathbf{V}(\mathbf{V}^{-1}\mathbf{x}) = \mathbf{x}$$

You just get $\mathbf{x}$ back. You went to Paris and came home without buying anything.

The real question is: **what does $\mathbf{A}$ do to a vector?**

Can we describe $\mathbf{A}$'s action using eigen-coordinates?

Let's find out by watching $\mathbf{A}$ act on a vector, *expressed in eigenvector language*.

What Does $\mathbf{A}$ Actually Do in Eigen-Coordinates?

Take $\mathbf{x} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$. In eigen-coordinates: $\mathbf{x} = 2\mathbf{v}_1 + 1\mathbf{v}_2$.

Now apply $\mathbf{A}$ and use linearity (distribute the multiplication):

$$\mathbf{Ax} = \mathbf{A}(2\mathbf{v}_1 + 1\mathbf{v}_2) = 2\mathbf{Av}_1 + 1\mathbf{Av}_2$$

But we **know** what $\mathbf{A}$ does to eigenvectors! $\mathbf{Av}_i = \lambda_i\mathbf{v}_i$.

$$= 2(\lambda_1\mathbf{v}_1) + 1(\lambda_2\mathbf{v}_2) = 2(3\mathbf{v}_1) + 1(1\mathbf{v}_2) = 6\mathbf{v}_1 + 1\mathbf{v}_2$$

Look at what happened to the eigen-coordinates:

$$\text{Before } \mathbf{A}: \begin{bmatrix} 2 \\ 1 \end{bmatrix}_{\text{eigen}} \quad \text{After } \mathbf{A}: \begin{bmatrix} 6 \\ 1 \end{bmatrix}_{\text{eigen}}$$

The $\mathbf{v}_1$ -component was multiplied by $\lambda_1 = 3$: $2 \rightarrow 6$.

The $\mathbf{v}_2$ -component was multiplied by $\lambda_2 = 1$: $1 \rightarrow 1$.

Each component was scaled **independently** by its eigenvalue. No mixing!

We just discovered: in eigen-coordinates, **A**'s action is:

“multiply the first component by λ_1 , the second by λ_2 .”

What matrix does exactly that? A **diagonal** matrix!

$$\begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \end{bmatrix} = \begin{bmatrix} \lambda_1 c_1 \\ \lambda_2 c_2 \end{bmatrix} = \begin{bmatrix} 3 c_1 \\ 1 \cdot c_2 \end{bmatrix}$$

We give this matrix a name: $\mathbf{\Lambda} = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}$.

$\mathbf{\Lambda}$ is not some arbitrary matrix we “introduced.”

Why is it diagonal? Each eigenvector direction is affected independently — that independence is exactly what “diagonal” means.

Deriving $\mathbf{A} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1}$ (Not Just Stating It)

Start from what we know: $\mathbf{A}\mathbf{v}_i = \lambda_i\mathbf{v}_i$ for each eigenvector.

Write both eigenvector equations side by side as a **single matrix equation**:

$$\mathbf{A} \begin{bmatrix} \mathbf{v}_1 & \mathbf{v}_2 \end{bmatrix} = \begin{bmatrix} \lambda_1\mathbf{v}_1 & \lambda_2\mathbf{v}_2 \end{bmatrix}$$

The left side is $\mathbf{A}\mathbf{V}$.

The right side can be split:
$$\begin{bmatrix} \lambda_1\mathbf{v}_1 & \lambda_2\mathbf{v}_2 \end{bmatrix} = \begin{bmatrix} \mathbf{v}_1 & \mathbf{v}_2 \end{bmatrix} \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} = \mathbf{V}\mathbf{\Lambda}$$

So we have: $\mathbf{A}\mathbf{V} = \mathbf{V}\mathbf{\Lambda}$

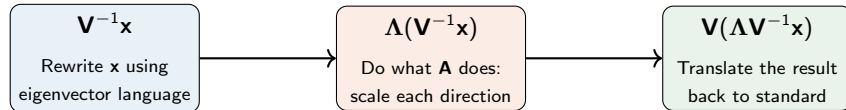
Multiply both sides on the right by $\mathbf{V}^{-1}$:

$$\mathbf{A}\mathbf{V}\mathbf{V}^{-1} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1} \implies \boxed{\mathbf{A} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1}}$$

This is the **eigenvalue decomposition**. It came from $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$ — we didn't pull it from thin air.

Reading $\mathbf{A} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1}$ as a Story

Now the formula tells a clear story. Read $\mathbf{Ax} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1}\mathbf{x}$ from **right to left**:



Why is the middle step needed?

Because $\mathbf{A}$ *does* something to vectors — it transforms them.

$\mathbf{V}^{-1}$ and $\mathbf{V}$ are just translators; $\mathbf{\Lambda}$ is *the actual work* that $\mathbf{A}$ performs, expressed in the coordinate system where that work looks simplest.

Let's Walk Through It With Numbers

$$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}, \quad \mathbf{V} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad \mathbf{V}^{-1} = \begin{bmatrix} 1/2 & 1/2 \\ 1/2 & -1/2 \end{bmatrix}, \quad \mathbf{\Lambda} = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}$$

Step 1 (translate to eigen-language):

$$\mathbf{V}^{-1}\mathbf{x} = \begin{bmatrix} 1/2 & 1/2 \\ 1/2 & -1/2 \end{bmatrix} \begin{bmatrix} 3 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

“ $\mathbf{x}$ is 2 parts $\mathbf{v}_1$, 1 part $\mathbf{v}_2$.”

Step 2 (do what $\mathbf{A}$ does — scale each component by its eigenvalue):

$$\mathbf{\Lambda} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \times 2 \\ 1 \times 1 \end{bmatrix} = \begin{bmatrix} 6 \\ 1 \end{bmatrix}$$

“Now 6 parts $\mathbf{v}_1$, still 1 part $\mathbf{v}_2$.”

Step 3 (translate back to standard coordinates):

$$\mathbf{V} \begin{bmatrix} 6 \\ 1 \end{bmatrix} = 6 \begin{bmatrix} 1 \\ 1 \end{bmatrix} + 1 \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 7 \\ 5 \end{bmatrix}$$

Verify directly: $\mathbf{A} \begin{bmatrix} 3 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 3 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \cdot 3 + 1 \cdot 1 \\ 1 \cdot 3 + 2 \cdot 1 \end{bmatrix} = \begin{bmatrix} 7 \\ 5 \end{bmatrix} \quad \checkmark$

Both paths give $\begin{bmatrix} 7 \\ 5 \end{bmatrix}$. The decomposition didn't change the answer — it revealed that $\mathbf{A}$ just tripled the $\mathbf{v}_1$ -part ($\lambda_1 = 3$) and left the $\mathbf{v}_2$ -part alone ($\lambda_2 = 1$).

“So Why Not Just Multiply Ax Directly?”

For a single multiplication, there's no computational advantage.

But what if you need $A^{100}x$? That's 100 matrix multiplications.

Eigenvalue decomposition makes this trivial.

Let's build it step by step, starting with A^2 .

Deriving $\mathbf{A}^k = \mathbf{V}\mathbf{\Lambda}^k\mathbf{V}^{-1}$

$\mathbf{A}^2 = \mathbf{A} \cdot \mathbf{A}$. Replace each $\mathbf{A}$ with $\mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1}$:

$$\mathbf{A}^2 = (\mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1})(\mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1}) = \mathbf{V}\mathbf{\Lambda} \underbrace{(\mathbf{V}^{-1}\mathbf{V})}_{\mathbf{I}} \mathbf{\Lambda}\mathbf{V}^{-1} = \mathbf{V}\mathbf{\Lambda}^2\mathbf{V}^{-1}$$

The $\mathbf{V}^{-1}$ from the first copy cancels with the $\mathbf{V}$ from the second! The two “translation” steps in the middle annihilate each other.

$$\mathbf{A}^3 = \mathbf{A}^2 \cdot \mathbf{A} = (\mathbf{V}\mathbf{\Lambda}^2\mathbf{V}^{-1})(\mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1}) = \mathbf{V}\mathbf{\Lambda}^2 \underbrace{(\mathbf{V}^{-1}\mathbf{V})}_{\mathbf{I}} \mathbf{\Lambda}\mathbf{V}^{-1} = \mathbf{V}\mathbf{\Lambda}^3\mathbf{V}^{-1}$$

The pattern is clear:

$$\boxed{\mathbf{A}^k = \mathbf{V}\mathbf{\Lambda}^k\mathbf{V}^{-1}}$$

You only translate once at the start ($\mathbf{V}^{-1}$) and once at the end ($\mathbf{V}$). All 100 multiplications happen as $\mathbf{\Lambda}^{100}$ — in the easy coordinate system.

Why Λ^{100} Is Trivial

Λ is diagonal, so raising it to a power just raises each entry:

$$\Lambda^2 = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 3 \cdot 3 & 0 \\ 0 & 1 \cdot 1 \end{bmatrix} = \begin{bmatrix} 9 & 0 \\ 0 & 1 \end{bmatrix}$$

$$\Lambda^3 = \begin{bmatrix} 27 & 0 \\ 0 & 1 \end{bmatrix}, \quad \Lambda^4 = \begin{bmatrix} 81 & 0 \\ 0 & 1 \end{bmatrix}, \quad \dots$$

$$\Lambda^{100} = \begin{bmatrix} 3^{100} & 0 \\ 0 & 1^{100} \end{bmatrix} = \begin{bmatrix} 3^{100} & 0 \\ 0 & 1 \end{bmatrix}$$

($3^{100} \approx 5.15 \times 10^{47}$. The $\mathbf{v}_1$ -direction gets amplified astronomically; the $\mathbf{v}_2$ -direction stays the same.)

The Power Method

The Power Method — Core Intuition

Question: Can we find eigenvalues *without* expanding determinants?

Key insight: Any vector $\mathbf{x}$ is a mix of eigenvectors:

$$\mathbf{x} = c_1 \mathbf{v}_1 + c_2 \mathbf{v}_2 + \cdots + c_n \mathbf{v}_n$$

When we multiply by $\mathbf{A}$:

$$\mathbf{A}\mathbf{x} = c_1 \lambda_1 \mathbf{v}_1 + c_2 \lambda_2 \mathbf{v}_2 + \cdots + c_n \lambda_n \mathbf{v}_n$$

Multiply k times:

$$\mathbf{A}^k \mathbf{x} = c_1 \lambda_1^k \mathbf{v}_1 + c_2 \lambda_2^k \mathbf{v}_2 + \cdots + c_n \lambda_n^k \mathbf{v}_n$$

If $|\lambda_1| > |\lambda_2| \geq \cdots$, then λ_1^k grows **much faster** than the rest!

After many multiplications, the $\mathbf{v}_1$ direction **dominates**. Everything else fades away.

Why the Largest Eigenvalue Wins

Factor out λ_1^k :

$$\mathbf{A}^k \mathbf{x} = \lambda_1^k \left[c_1 \mathbf{v}_1 + c_2 \left(\frac{\lambda_2}{\lambda_1} \right)^k \mathbf{v}_2 + \cdots + c_n \left(\frac{\lambda_n}{\lambda_1} \right)^k \mathbf{v}_n \right]$$

Since $|\lambda_1| > |\lambda_i|$ for $i \geq 2$, each ratio $\left| \frac{\lambda_i}{\lambda_1} \right| < 1$.

Raise a number less than 1 to a large power $\rightarrow$ it approaches **zero**.

So as $k \rightarrow \infty$:

$$\mathbf{A}^k \mathbf{x} \approx \lambda_1^k c_1 \mathbf{v}_1$$

The result points in the direction of $\mathbf{v}_1$ (the dominant eigenvector).
The scaling factor gives us λ_1 (the dominant eigenvalue).

The Power Method: Steps

1. Start with any nonzero vector $\mathbf{x}_0$.
2. Multiply: $\mathbf{y}_{k+1} = \mathbf{A} \mathbf{x}_k$
3. Normalize: divide $\mathbf{y}_{k+1}$ by its largest-magnitude entry to get $\mathbf{x}_{k+1}$
The **entry you divided by** is your eigenvalue estimate.
The **resulting vector** is your eigenvector estimate.
4. Repeat steps 2–3 until the eigenvalue estimate stops changing.

Two questions arise:

- (a) Why do we normalize at all?
- (b) Why does the normalizing factor give us the eigenvalue?

Both have the same answer, which the next slide will make precise.

Why Does the Normalizing Factor Give λ ?

Suppose after several iterations, $\mathbf{x}_k$ is close to the true eigenvector $\mathbf{v}_1$. After normalization, its largest entry is 1.

What happens when we multiply by $\mathbf{A}$? Since $\mathbf{x}_k \approx \mathbf{v}_1$, and $\mathbf{A}\mathbf{v}_1 = \lambda_1\mathbf{v}_1$:

$$\mathbf{y}_{k+1} = \mathbf{A}\mathbf{x}_k \approx \lambda_1\mathbf{x}_k$$

Every entry of $\mathbf{y}_{k+1}$ is approximately λ_1 times the corresponding entry of $\mathbf{x}_k$.

The largest entry of $\mathbf{x}_k$ was 1 (we normalized it). So the largest entry of $\mathbf{y}_{k+1} \approx \lambda_1 \times 1 = \lambda_1$.

When we normalize $\mathbf{y}_{k+1}$ by dividing out its largest entry, that entry is $\approx \lambda_1$.

The normalizing factor IS the eigenvalue estimate.

The division keeps $\mathbf{x}_{k+1}$'s largest entry at 1, preventing the numbers from exploding as λ_1^k .

Before convergence, $\mathbf{x}_k$ is only approximately $\mathbf{v}_1$, so the normalizing factor is only approximately λ_1 .

But it gets closer to λ_1 with every iteration.

Power Method: a 2×2 Example

$$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}, \quad \text{eigenvectors: } \mathbf{v}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \ (\lambda_1 = 3), \quad \mathbf{v}_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix} \ (\lambda_2 = 1)$$

$$\text{Start: } \mathbf{x}_0 = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad (\text{deliberately } \textit{not} \text{ an eigenvector})$$

$$\text{Iteration 1: } \mathbf{y}_1 = \mathbf{A} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

$$\text{Largest entry: } \mathbf{2}. \quad \text{Divide: } \mathbf{x}_1 = \begin{bmatrix} 1 \\ 0.5 \end{bmatrix}. \quad \lambda \approx \mathbf{2}$$

If $\mathbf{x}_0$ were the eigenvector, this entry would be exactly 3. We're not there yet.

Power Method: a 2×2 Example

$$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}, \quad \text{eigenvectors: } \mathbf{v}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix} (\lambda_1 = 3), \quad \mathbf{v}_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix} (\lambda_2 = 1)$$

$$\text{Start: } \mathbf{x}_0 = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad (\text{deliberately } \textit{not} \text{ an eigenvector})$$

$$\text{Iteration 2: } \mathbf{y}_2 = \mathbf{A} \begin{bmatrix} 1 \\ 0.5 \end{bmatrix} = \begin{bmatrix} 2.5 \\ 2 \end{bmatrix}$$

$$\text{Largest entry: } \mathbf{2.5}. \quad \text{Divide: } \mathbf{x}_2 = \begin{bmatrix} 1 \\ 0.8 \end{bmatrix}. \quad \lambda \approx \mathbf{2.5}$$

Getting closer to 3. The vector is getting closer to $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$.

Power Method: a 2×2 Example

$$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}, \quad \text{eigenvectors: } \mathbf{v}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix} (\lambda_1 = 3), \quad \mathbf{v}_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix} (\lambda_2 = 1)$$

$$\text{Start: } \mathbf{x}_0 = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad (\text{deliberately } \textit{not} \text{ an eigenvector})$$

$$\text{Iteration 3: } \mathbf{y}_3 = \mathbf{A} \begin{bmatrix} 1 \\ 0.8 \end{bmatrix} = \begin{bmatrix} 2.8 \\ 2.6 \end{bmatrix}$$

$$\text{Largest entry: } \mathbf{2.8}. \quad \text{Divide: } \mathbf{x}_3 = \begin{bmatrix} 1 \\ 0.929 \end{bmatrix}. \quad \lambda \approx \mathbf{2.8}$$

Almost there! As the vector approaches $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$, the normalizing factor approaches 3.

Power Method: a 2×2 Example

$$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}, \quad \text{eigenvectors: } \mathbf{v}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix} (\lambda_1 = 3), \quad \mathbf{v}_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix} (\lambda_2 = 1)$$

$$\text{Start: } \mathbf{x}_0 = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad (\text{deliberately } \textit{not} \text{ an eigenvector})$$

$$\text{Iteration 4: } \mathbf{y}_4 = \mathbf{A} \begin{bmatrix} 1 \\ 0.929 \end{bmatrix} = \begin{bmatrix} 2.929 \\ 2.857 \end{bmatrix}$$

$$\text{Largest entry: } \mathbf{2.929}. \quad \text{Divide: } \mathbf{x}_4 \approx \begin{bmatrix} 1 \\ 0.976 \end{bmatrix}. \quad \lambda \approx \mathbf{2.929}$$

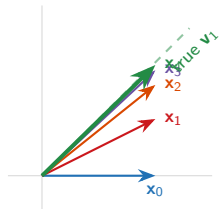
$$\text{When } \mathbf{x}_k = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \text{ exactly, the result would be } \begin{bmatrix} 3 \\ 3 \end{bmatrix} \text{ and the factor would be exactly } 3.$$

Power Method: a 2×2 Example

$$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}, \quad \text{eigenvectors: } \mathbf{v}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix} (\lambda_1 = 3), \quad \mathbf{v}_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix} (\lambda_2 = 1)$$

Iter	$\mathbf{x}_k$	Normalizing factor $\approx \lambda_k$	ϵ_a	Why it approximates λ_1
0	$\begin{bmatrix} 1 \\ 0 \end{bmatrix}$	—	—	—
1	$\begin{bmatrix} 1 \\ 0.500 \end{bmatrix}$	2.000	—	$\mathbf{x}_0$ far from $\mathbf{v}_1$
2	$\begin{bmatrix} 1 \\ 0.800 \end{bmatrix}$	2.500	20.000%	getting closer
3	$\begin{bmatrix} 1 \\ 0.929 \end{bmatrix}$	2.800	10.714%	closer still
4	$\begin{bmatrix} 1 \\ 0.976 \end{bmatrix}$	2.929	4.404%	almost there
$\vdots$	$\vdots$	$\vdots$	$\vdots$	
∞	$\begin{bmatrix} 1 \\ 1 \end{bmatrix}$	3.000	—%	$\mathbf{x}_k = \mathbf{v}_1 \Rightarrow \text{factor} = \lambda_1$ exactly

Watching the Vector Converge



The vector “swings” from $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$ toward the eigenvector direction $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$.

Each iteration, the $\mathbf{v}_2$ -component shrinks by a factor of $\lambda_2/\lambda_1 = 1/3$.

Yet another example

$$\mathbf{A} = \begin{bmatrix} 3.556 & -1.778 & 0 \\ -1.778 & 3.556 & -1.778 \\ 0 & -1.778 & 3.556 \end{bmatrix}$$

$$\text{Start: } \mathbf{x}_0 = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

$$\text{Iteration 1: } \mathbf{A} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1.778 \\ 0 \\ 1.778 \end{bmatrix} = 1.778 \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} \quad \lambda \approx \mathbf{1.778}$$

Yet another example

$$\mathbf{A} = \begin{bmatrix} 3.556 & -1.778 & 0 \\ -1.778 & 3.556 & -1.778 \\ 0 & -1.778 & 3.556 \end{bmatrix}$$

$$\text{Start: } \mathbf{x}_0 = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

$$\text{Iteration 2: } \mathbf{A} \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 3.556 \\ -3.556 \\ 3.556 \end{bmatrix} = 3.556 \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix} \quad \lambda \approx \mathbf{3.556} \quad |\varepsilon_a| = 50\%$$

Yet another example

$$\mathbf{A} = \begin{bmatrix} 3.556 & -1.778 & 0 \\ -1.778 & 3.556 & -1.778 \\ 0 & -1.778 & 3.556 \end{bmatrix}$$

$$\text{Start: } \mathbf{x}_0 = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

$$\text{Iteration 3: } \mathbf{A} \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix} = \begin{bmatrix} 5.334 \\ -7.112 \\ 5.334 \end{bmatrix} = -7.112 \begin{bmatrix} -0.75 \\ 1 \\ -0.75 \end{bmatrix} \quad \lambda \approx -\mathbf{7.112}$$

(Sign change $\rightarrow$ inflated error. Don't panic.)

Yet another example

$$\mathbf{A} = \begin{bmatrix} 3.556 & -1.778 & 0 \\ -1.778 & 3.556 & -1.778 \\ 0 & -1.778 & 3.556 \end{bmatrix}$$

$$\text{Start: } \mathbf{x}_0 = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

$$\text{Iteration 4: } \mathbf{A} \begin{bmatrix} -0.75 \\ 1 \\ -0.75 \end{bmatrix} = \begin{bmatrix} -4.445 \\ 6.223 \\ -4.445 \end{bmatrix} = 6.223 \begin{bmatrix} -0.714 \\ 1 \\ -0.714 \end{bmatrix} \quad \lambda \approx \mathbf{6.223}$$

Yet another example

$$\mathbf{A} = \begin{bmatrix} 3.556 & -1.778 & 0 \\ -1.778 & 3.556 & -1.778 \\ 0 & -1.778 & 3.556 \end{bmatrix}$$

$$\text{Start: } \mathbf{x}_0 = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

$$\text{Iteration 5: } \mathbf{A} \begin{bmatrix} -0.714 \\ 1 \\ -0.714 \end{bmatrix} = \begin{bmatrix} -4.317 \\ 6.095 \\ -4.317 \end{bmatrix} = 6.095 \begin{bmatrix} -0.708 \\ 1 \\ -0.708 \end{bmatrix} \quad \lambda \approx \mathbf{6.095}$$

Converging toward the true value $\lambda_{\max} \approx 6.070$.

Yet another example

$$|\varepsilon_a| = \left| \frac{\lambda_k - \lambda_{k-1}}{\lambda_k} \right| \times 100\%$$

Iter	$\mathbf{x}_k$	Normalizing factor $\approx \lambda_k$	$ \varepsilon_a $
0	$[1.00000, 1.00000, 1.00000]^T$	—	—
1	$[1.00000, 0.00000, 1.00000]^T$	1.77800	—
2	$[1.00000, -1.00000, 1.00000]^T$	3.55600	50.00000%
3	$[-0.75000, 1.00000, -0.75000]^T$	-7.11200	150.00000%
4	$[-0.71429, 1.00000, -0.71429]^T$	6.22300	214.28571%
5	$[-0.70833, 1.00000, -0.70833]^T$	6.09600	2.08333%
6	$[-0.70732, 1.00000, -0.70732]^T$	6.07483	0.34843%
7	$[-0.70714, 1.00000, -0.70714]^T$	6.07122	0.05952%
8	$[-0.70711, 1.00000, -0.70711]^T$	6.07060	0.01021%
9	$[-0.70711, 1.00000, -0.70711]^T$	6.07049	0.00175%
10	$[-0.70711, 1.00000, -0.70711]^T$	6.07048	0.00030%

The large errors at Iterations 3–4 occur because the normalizing factor changes sign. After that, the estimates settle toward $\lambda_{\max} \approx 6.07048$.

Smallest & Intermediate Eigenvalues

A Concrete 4×4 Matrix to Work With

$$\mathbf{A} = \begin{bmatrix} 8 & 2 & 0 & 0 \\ 2 & 8 & 0 & 0 \\ 0 & 0 & 3 & 1 \\ 0 & 0 & 1 & 3 \end{bmatrix}$$

Let's find its eigenvalues the “old-fashioned” way: $\det(\mathbf{A} - \lambda \mathbf{I}) = 0$.

Characteristic polynomial: $(\lambda - 10)(\lambda - 6)(\lambda - 4)(\lambda - 2) = 0$

Eigenvalues: $\lambda_1 = 10$, $\lambda_2 = 6$, $\lambda_3 = 4$, $\lambda_4 = 2$.

Let's Find the Eigenvectors

Check $\lambda_1 = 10$: Solve $(\mathbf{A} - 10\mathbf{I})\mathbf{v} = \mathbf{0}$:

$$\begin{bmatrix} -2 & 2 & 0 & 0 \\ 2 & -2 & 0 & 0 \\ 0 & 0 & -7 & 1 \\ 0 & 0 & 1 & -7 \end{bmatrix} \mathbf{v} = \mathbf{0} \Rightarrow v_1 = v_2, \quad v_3 = v_4 = 0 \Rightarrow \mathbf{v}_1 = \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

$$\text{Verify: } \mathbf{A} \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 8+2 \\ 2+8 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 10 \\ 10 \\ 0 \\ 0 \end{bmatrix} = 10 \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \end{bmatrix} \quad \checkmark$$

$$\text{Similarly: } \mathbf{v}_2 = \begin{bmatrix} 1 \\ -1 \\ 0 \\ 0 \end{bmatrix} (\lambda_2 = 6), \quad \mathbf{v}_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 1 \end{bmatrix} (\lambda_3 = 4), \quad \mathbf{v}_4 = \begin{bmatrix} 0 \\ 0 \\ 1 \\ -1 \end{bmatrix} (\lambda_4 = 2)$$

We Found the Biggest. What About the Others?

If we run the power method on this **A**, it finds $\lambda_1 = 10$.

But what if we need $\lambda_4 = 2$ (the smallest)?

Or $\lambda_2 = 6$ (the second largest)?

The power method only knows how to find the **dominant** (largest-magnitude) eigenvalue.

The trick in both cases is the same: **transform the matrix** so that the eigenvalue we want *becomes* the dominant one, then run the power method on the transformed matrix.

Two transformations cover most needs: **inversion** (for the smallest) and **deflation** (for intermediates).

The Inverse Trick: Flipping Eigenvalues

Fact: If λ is an eigenvalue of $\mathbf{A}$, then $\frac{1}{\lambda}$ is an eigenvalue of $\mathbf{A}^{-1}$.

Proof (one line): $\mathbf{A}\mathbf{v} = \lambda\mathbf{v} \Rightarrow \mathbf{v} = \lambda\mathbf{A}^{-1}\mathbf{v} \Rightarrow \mathbf{A}^{-1}\mathbf{v} = \frac{1}{\lambda}\mathbf{v}$. ✓

What does flipping do to our matrix's eigenvalues?

Eigenvalues of $\mathbf{A} \longrightarrow$ Eigenvalues of $\mathbf{A}^{-1}$

$$\lambda_1 = 10 \longrightarrow 1/10 = 0.1 \quad (\text{was largest, now smallest})$$

$$\lambda_2 = 6 \longrightarrow 1/6 \approx 0.167$$

$$\lambda_3 = 4 \longrightarrow 1/4 = 0.25$$

$$\lambda_4 = 2 \longrightarrow 1/2 = \mathbf{0.5} \quad (\text{was smallest, now **largest!**})$$

Inversion **flips the ranking**: the smallest eigenvalue of $\mathbf{A}$ becomes the largest eigenvalue of $\mathbf{A}^{-1}$.

The power method on $\mathbf{A}^{-1}$ finds $0.5 = 1/\lambda_4$. Take the reciprocal $\rightarrow \lambda_4 = 2$.

Inverse Power Method

Goal: Find the smallest eigenvalue of $\mathbf{A}$.

Step 1: Start with any nonzero $\mathbf{x}_0$.

Step 2: Each iteration, compute $\mathbf{y}_{k+1} = \mathbf{A}^{-1}\mathbf{x}_k$.

(In practice: solve $\mathbf{A}\mathbf{y}_{k+1} = \mathbf{x}_k$ instead of computing $\mathbf{A}^{-1}$ directly.

Use LU decomposition — factor once, solve cheaply every iteration.)

Step 3: Normalize $\mathbf{y}_{k+1}$ by its largest entry to get $\mathbf{x}_{k+1}$. That entry $\approx 1/\lambda_{\min}$.

Step 4: Repeat until convergence. Take the reciprocal: $\lambda_{\min} \approx 1/(\text{normalizing factor})$.

Everything else is identical to the regular power method — we just swapped $\mathbf{A}$ for $\mathbf{A}^{-1}$.

Inverse Power Method – Example

$$\mathbf{A} = \begin{bmatrix} 8 & 2 & 0 & 0 \\ 2 & 8 & 0 & 0 \\ 0 & 0 & 3 & 1 \\ 0 & 0 & 1 & 3 \end{bmatrix}, \quad \mathbf{A}^{-1} = \begin{bmatrix} \frac{2}{15} & -\frac{1}{30} & 0 & 0 \\ -\frac{1}{30} & \frac{2}{15} & 0 & 0 \\ 0 & 0 & \frac{3}{8} & -\frac{1}{8} \\ 0 & 0 & -\frac{1}{8} & \frac{3}{8} \end{bmatrix}$$

Start with

$$\mathbf{x}_0 = \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

Iteration 1:

$$\mathbf{A}^{-1}\mathbf{x}_0 = \begin{bmatrix} 0.13333 \\ -0.03333 \\ 0.37500 \\ -0.12500 \end{bmatrix} = 0.37500 \begin{bmatrix} 0.35556 \\ -0.08889 \\ 1.00000 \\ -0.33333 \end{bmatrix}$$

$$\mathbf{x}_1 = \begin{bmatrix} 0.35556 \\ -0.08889 \\ 1.00000 \\ -0.33333 \end{bmatrix}, \quad \lambda_{\min} \approx \frac{1}{0.37500} = 2.66667$$

Inverse Power Method – Example

$$\mathbf{A} = \begin{bmatrix} 8 & 2 & 0 & 0 \\ 2 & 8 & 0 & 0 \\ 0 & 0 & 3 & 1 \\ 0 & 0 & 1 & 3 \end{bmatrix}, \quad \mathbf{A}^{-1} = \begin{bmatrix} \frac{2}{15} & -\frac{1}{30} & 0 & 0 \\ -\frac{1}{30} & \frac{2}{15} & 0 & 0 \\ 0 & 0 & \frac{3}{8} & -\frac{1}{8} \\ 0 & 0 & -\frac{1}{8} & \frac{3}{8} \end{bmatrix}$$

Start with

$$\mathbf{x}_0 = \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

Iteration 2:

$$\mathbf{A}^{-1}\mathbf{x}_1 = \begin{bmatrix} 0.05037 \\ -0.02370 \\ 0.41667 \\ -0.25000 \end{bmatrix} = 0.41667 \begin{bmatrix} 0.12089 \\ -0.05689 \\ 1.00000 \\ -0.60000 \end{bmatrix}$$

$$\mathbf{x}_2 = \begin{bmatrix} 0.12089 \\ -0.05689 \\ 1.00000 \\ -0.60000 \end{bmatrix}, \quad \lambda_{\min} \approx \frac{1}{0.41667} = 2.40000$$

Inverse Power Method – Example

$$\mathbf{A} = \begin{bmatrix} 8 & 2 & 0 & 0 \\ 2 & 8 & 0 & 0 \\ 0 & 0 & 3 & 1 \\ 0 & 0 & 1 & 3 \end{bmatrix}, \quad \mathbf{A}^{-1} = \begin{bmatrix} \frac{2}{15} & -\frac{1}{30} & 0 & 0 \\ -\frac{1}{30} & \frac{2}{15} & 0 & 0 \\ 0 & 0 & \frac{3}{8} & -\frac{1}{8} \\ 0 & 0 & -\frac{1}{8} & \frac{3}{8} \end{bmatrix}$$

Start with

$$\mathbf{x}_0 = \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

Iteration 3:

$$\mathbf{A}^{-1}\mathbf{x}_2 = \begin{bmatrix} 0.01801 \\ -0.01161 \\ 0.45000 \\ -0.35000 \end{bmatrix} = 0.45000 \begin{bmatrix} 0.04003 \\ -0.02581 \\ 1.00000 \\ -0.77778 \end{bmatrix}$$

$$\mathbf{x}_3 = \begin{bmatrix} 0.04003 \\ -0.02581 \\ 1.00000 \\ -0.77778 \end{bmatrix}, \quad \lambda_{\min} \approx \frac{1}{0.45000} = 2.22222$$

Inverse Power Method – Example

$$\mathbf{A} = \begin{bmatrix} 8 & 2 & 0 & 0 \\ 2 & 8 & 0 & 0 \\ 0 & 0 & 3 & 1 \\ 0 & 0 & 1 & 3 \end{bmatrix}, \quad \mathbf{A}^{-1} = \begin{bmatrix} \frac{2}{15} & -\frac{1}{30} & 0 & 0 \\ -\frac{1}{30} & \frac{2}{15} & 0 & 0 \\ 0 & 0 & \frac{3}{8} & -\frac{1}{8} \\ 0 & 0 & -\frac{1}{8} & \frac{3}{8} \end{bmatrix}$$

Start with

$$\mathbf{x}_0 = \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

Iteration 4:

$$\mathbf{A}^{-1}\mathbf{x}_3 = \begin{bmatrix} 0.00620 \\ -0.00478 \\ 0.47222 \\ -0.41667 \end{bmatrix} = 0.47222 \begin{bmatrix} 0.01313 \\ -0.01011 \\ 1.00000 \\ -0.88235 \end{bmatrix}$$

$$\mathbf{x}_4 = \begin{bmatrix} 0.01313 \\ -0.01011 \\ 1.00000 \\ -0.88235 \end{bmatrix}, \quad \lambda_{\min} \approx \frac{1}{0.47222} = 2.11765$$

Inverse Power Method – Example

$$\mathbf{A} = \begin{bmatrix} 8 & 2 & 0 & 0 \\ 2 & 8 & 0 & 0 \\ 0 & 0 & 3 & 1 \\ 0 & 0 & 1 & 3 \end{bmatrix}, \quad \mathbf{A}^{-1} = \begin{bmatrix} \frac{2}{15} & -\frac{1}{30} & 0 & 0 \\ -\frac{1}{30} & \frac{2}{15} & 0 & 0 \\ 0 & 0 & \frac{3}{8} & -\frac{1}{8} \\ 0 & 0 & -\frac{1}{8} & \frac{3}{8} \end{bmatrix}$$

Start with

$$\mathbf{x}_0 = \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

Iteration 5:

$$\mathbf{A}^{-1}\mathbf{x}_4 = \begin{bmatrix} 0.00209 \\ -0.00179 \\ 0.48529 \\ -0.45588 \end{bmatrix} = 0.48529 \begin{bmatrix} 0.00430 \\ -0.00368 \\ 1.00000 \\ -0.93939 \end{bmatrix}$$

$$\mathbf{x}_5 = \begin{bmatrix} 0.00430 \\ -0.00368 \\ 1.00000 \\ -0.93939 \end{bmatrix}, \quad \lambda_{\min} \approx \frac{1}{0.48529} = 2.06061$$

Inverse Power Method – Example

$$\mathbf{A} = \begin{bmatrix} 8 & 2 & 0 & 0 \\ 2 & 8 & 0 & 0 \\ 0 & 0 & 3 & 1 \\ 0 & 0 & 1 & 3 \end{bmatrix}, \quad \mathbf{A}^{-1} = \begin{bmatrix} \frac{2}{15} & -\frac{1}{30} & 0 & 0 \\ -\frac{1}{30} & \frac{2}{15} & 0 & 0 \\ 0 & 0 & \frac{3}{8} & -\frac{1}{8} \\ 0 & 0 & -\frac{1}{8} & \frac{3}{8} \end{bmatrix}$$

Start with

$$\mathbf{x}_0 = \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

Iteration 6:

$$\mathbf{A}^{-1}\mathbf{x}_5 = \begin{bmatrix} 0.00070 \\ -0.00063 \\ 0.49242 \\ -0.47727 \end{bmatrix} = 0.49242 \begin{bmatrix} 0.00141 \\ -0.00129 \\ 1.00000 \\ -0.96923 \end{bmatrix}$$

$$\mathbf{x}_6 = \begin{bmatrix} 0.00141 \\ -0.00129 \\ 1.00000 \\ -0.96923 \end{bmatrix}, \quad \lambda_{\min} \approx \frac{1}{0.49242} = 2.03077$$

Inverse Power Method – Tabular Summary

$$|\varepsilon_a| = \left| \frac{\lambda_k - \lambda_{k-1}}{\lambda_k} \right| \times 100\%, \quad \lambda_k \approx \frac{1}{\text{normalizing factor}}$$

Iter	$\mathbf{x}_k$	Normalizing factor	$\lambda_{\min} \approx 1/\text{factor}$	$ \varepsilon_a $
0	$[1.00000, 0.00000, 1.00000, 0.00000]^T$	—	—	—
1	$[0.35556, -0.08889, 1.00000, -0.33333]^T$	0.37500	2.66667	—
2	$[0.12089, -0.05689, 1.00000, -0.60000]^T$	0.41667	2.40000	11.11111%
3	$[0.04003, -0.02581, 1.00000, -0.77778]^T$	0.45000	2.22222	8.00000%
4	$[0.01313, -0.01011, 1.00000, -0.88235]^T$	0.47222	2.11765	4.93827%
5	$[0.00430, -0.00368, 1.00000, -0.93939]^T$	0.48529	2.06061	2.76817%
6	$[0.00141, -0.00129, 1.00000, -0.96923]^T$	0.49242	2.03077	1.46924%

Since the dominant eigenvalue of $\mathbf{A}^{-1}$ is $\frac{1}{2} = 0.5$, the normalizing factor approaches 0.5, and therefore $\lambda_{\min}$ approaches 2.

Finding Intermediate Eigenvalues: The Problem

For our 4×4 matrix we found:

$$\lambda_1 = 10 \text{ (power method on } \mathbf{A})$$

$$\lambda_4 = 2 \text{ (power method on } \mathbf{A}^{-1})$$

But what about $\lambda_2 = 6$ and $\lambda_3 = 4$, stuck in the middle?

The power method always finds the *loudest* eigenvalue.

$\lambda_1 = 10$ drowns out $\lambda_2 = 6$.

What if we could turn down λ_1 's volume to zero?

Then $\lambda_2 = 6$ would become the loudest, and the power method would find it.

Let's figure out how to do this with our familiar 2×2 example.

Recall: What **A** Does, Broken Into Parts

Back to our 2×2 friend: $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$, $\lambda_1 = 3$, $\mathbf{v}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$; $\lambda_2 = 1$, $\mathbf{v}_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$.

Take $\mathbf{x} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$. We showed earlier that $\mathbf{x} = 2\mathbf{v}_1 + 1\mathbf{v}_2$.

When **A** acts on **x**, it scales each eigenvector part independently:

$$\mathbf{Ax} = \underbrace{3 \times 2\mathbf{v}_1}_{\mathbf{v}_1\text{-contribution}} + \underbrace{1 \times 1\mathbf{v}_2}_{\mathbf{v}_2\text{-contribution}} = \underbrace{\begin{bmatrix} 6 \\ 6 \end{bmatrix}}_{\text{from } \mathbf{v}_1} + \underbrace{\begin{bmatrix} 1 \\ -1 \end{bmatrix}}_{\text{from } \mathbf{v}_2} = \begin{bmatrix} 7 \\ 5 \end{bmatrix}$$

The result $\begin{bmatrix} 7 \\ 5 \end{bmatrix}$ is a **sum of two contributions**:

$\begin{bmatrix} 6 \\ 6 \end{bmatrix}$: what $\lambda_1 = 3$ did to the $\mathbf{v}_1$ -part of **x**

$\begin{bmatrix} 1 \\ -1 \end{bmatrix}$: what $\lambda_2 = 1$ did to the $\mathbf{v}_2$ -part of **x**

“Muting” λ_1 : Subtract Its Contribution

$$\mathbf{A} \begin{bmatrix} 3 \\ 1 \end{bmatrix} = \begin{bmatrix} 7 \\ 5 \end{bmatrix} = \underbrace{\begin{bmatrix} 6 \\ 6 \end{bmatrix}}_{\mathbf{v}_1\text{-part}} + \underbrace{\begin{bmatrix} 1 \\ -1 \end{bmatrix}}_{\mathbf{v}_2\text{-part}}$$

If we **subtract the $\mathbf{v}_1$ -contribution** from the result:

$$\begin{bmatrix} 7 \\ 5 \end{bmatrix} - \begin{bmatrix} 6 \\ 6 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

Only the $\mathbf{v}_2$ -part survives!

$$\text{And } \begin{bmatrix} 1 \\ -1 \end{bmatrix} = 1 \times \mathbf{v}_2 = \lambda_2 \times (\mathbf{v}_2\text{-component of } \mathbf{x}).$$

So if we had a matrix $\mathbf{A}_2$ that *automatically* subtracts the $\mathbf{v}_1$ -contribution from whatever $\mathbf{A}$ produces. . .

$\Rightarrow$ $\mathbf{A}_2$ would behave as if λ_1 doesn't exist.

$\Rightarrow$ The power method on $\mathbf{A}_2$ would find λ_2 .

Question: How do we compute “the $\mathbf{v}_1$ -contribution” for *any* input $\mathbf{x}$, not just $\begin{bmatrix} 3 \\ 1 \end{bmatrix}$?

Extracting a Component: The Dot Product Trick

We need two things: (a) find *how much* of $\mathbf{x}$ lies along $\mathbf{v}_1$, and (b) rebuild that piece as a vector.

Step (a): Use a *unit-length* version of $\mathbf{v}_1$: $\hat{\mathbf{v}}_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$

The dot product $\hat{\mathbf{v}}_1^T \mathbf{x}$ measures how much of $\mathbf{x}$ points along $\hat{\mathbf{v}}_1$:

$$\hat{\mathbf{v}}_1^T \begin{bmatrix} 3 \\ 1 \end{bmatrix} = \frac{1}{\sqrt{2}}(3+1) = \frac{4}{\sqrt{2}} = 2\sqrt{2} \quad \leftarrow \text{“amount of } \mathbf{x} \text{ along } \hat{\mathbf{v}}_1\text{”}$$

Step (b): Multiply $\hat{\mathbf{v}}_1$ by that amount to rebuild the component:

$$\hat{\mathbf{v}}_1 \times 2\sqrt{2} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \times 2\sqrt{2} = 2 \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \end{bmatrix}$$

This is exactly $2\mathbf{v}_1$, the $\mathbf{v}_1$ -component of $\mathbf{x}$!

$\hat{\mathbf{v}}_1(\hat{\mathbf{v}}_1^T \mathbf{x})$ extracts the $\mathbf{v}_1$ -part of any vector $\mathbf{x}$.

It is called the **projection** of $\mathbf{x}$ onto $\hat{\mathbf{v}}_1$.

Building the “Muted” Matrix $\mathbf{A}_2$

Now we can compute the $\mathbf{v}_1$ -contribution to $\mathbf{Ax}$ for *any* $\mathbf{x}$:

- $\hat{\mathbf{v}}_1(\hat{\mathbf{v}}_1^T \mathbf{x})$ gives the $\mathbf{v}_1$ -part of $\mathbf{x}$ (we just showed this)
- $\mathbf{A}$ scales that part by λ_1
- So the $\mathbf{v}_1$ -contribution in $\mathbf{Ax}$ is: $\lambda_1 \cdot \hat{\mathbf{v}}_1(\hat{\mathbf{v}}_1^T \mathbf{x})$

Subtract it from $\mathbf{Ax}$:

$$\mathbf{A}_2 \mathbf{x} = \mathbf{Ax} - \lambda_1 \hat{\mathbf{v}}_1(\hat{\mathbf{v}}_1^T \mathbf{x}) = (\mathbf{A} - \lambda_1 \hat{\mathbf{v}}_1 \hat{\mathbf{v}}_1^T) \mathbf{x}$$

So the deflated matrix is: $\mathbf{A}_2 = \mathbf{A} - \lambda_1 \hat{\mathbf{v}}_1 \hat{\mathbf{v}}_1^T$

Let's compute it for our 2×2 example:

$$\hat{\mathbf{v}}_1 \hat{\mathbf{v}}_1^T = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \cdot \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \quad \lambda_1 \hat{\mathbf{v}}_1 \hat{\mathbf{v}}_1^T = \frac{3}{2} \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 1.5 & 1.5 \\ 1.5 & 1.5 \end{bmatrix}$$

$$\mathbf{A}_2 = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} - \begin{bmatrix} 1.5 & 1.5 \\ 1.5 & 1.5 \end{bmatrix} = \begin{bmatrix} 0.5 & -0.5 \\ -0.5 & 0.5 \end{bmatrix}$$

Does It Work? Let's Check $\mathbf{A}_2$

$$\mathbf{A}_2 = \begin{bmatrix} 0.5 & -0.5 \\ -0.5 & 0.5 \end{bmatrix}$$

Test 1: Apply $\mathbf{A}_2$ to $\mathbf{v}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$: $\mathbf{A}_2 \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.5 - 0.5 \\ -0.5 + 0.5 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$

λ_1 has been replaced by **0**. The $\mathbf{v}_1$ -direction has been **muted**!

Test 2: Apply $\mathbf{A}_2$ to $\mathbf{v}_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$: $\mathbf{A}_2 \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 0.5 + 0.5 \\ -0.5 - 0.5 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \end{bmatrix} = 1 \cdot \begin{bmatrix} 1 \\ -1 \end{bmatrix}$

$\lambda_2 = 1$ is **unchanged**.

$\mathbf{A}_2$ has eigenvalues $\{0, 1\}$.

The power method on $\mathbf{A}_2$ finds $\lambda_2 = 1$. We successfully “muted” $\lambda_1 = 3$!

Why Does This Leave $\mathbf{v}_2$ Untouched?

The subtraction $\mathbf{A}_2 = \mathbf{A} - \lambda_1 \hat{\mathbf{v}}_1 \hat{\mathbf{v}}_1^T$ removes the $\mathbf{v}_1$ -part from the result.

But why doesn't it accidentally damage the $\mathbf{v}_2$ -part too?

Because $\hat{\mathbf{v}}_1$ and $\hat{\mathbf{v}}_2$ are **perpendicular** (orthogonal):

$$\hat{\mathbf{v}}_1^T \hat{\mathbf{v}}_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \end{bmatrix} \cdot \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \frac{1}{2}(1 + (-1)) = 0$$

When $\mathbf{A}_2$ acts on $\hat{\mathbf{v}}_2$:

$$\mathbf{A}_2 \hat{\mathbf{v}}_2 = \mathbf{A} \hat{\mathbf{v}}_2 - \lambda_1 \hat{\mathbf{v}}_1 \underbrace{(\hat{\mathbf{v}}_1^T \hat{\mathbf{v}}_2)}_{=0} = \mathbf{A} \hat{\mathbf{v}}_2 - \mathbf{0} = \lambda_2 \hat{\mathbf{v}}_2$$

The projection $\hat{\mathbf{v}}_1(\hat{\mathbf{v}}_1^T \hat{\mathbf{v}}_2)$ tries to find “how much of $\hat{\mathbf{v}}_2$ is along $\hat{\mathbf{v}}_1$.”

Answer: **zero**, because they're perpendicular. So nothing gets subtracted.

This is why Hotelling's deflation works for **symmetric** matrices — their eigenvectors are always orthogonal.

Deflation: Steps

Step 1: Use the power method on $\mathbf{A} \rightarrow$ get λ_1 and $\mathbf{v}_1$.

Step 2: Normalize $\mathbf{v}_1$ to unit length: $\hat{\mathbf{v}}_1 = \frac{\mathbf{v}_1}{\sqrt{\mathbf{v}_1^T \mathbf{v}_1}}$

Step 3: Compute the deflated matrix: $\mathbf{A}_2 = \mathbf{A} - \lambda_1 \hat{\mathbf{v}}_1 \hat{\mathbf{v}}_1^T$

(This “mutes” λ_1 by subtracting its contribution from everything $\mathbf{A}$ does.)

Step 4: Apply the power method to $\mathbf{A}_2 \rightarrow$ get λ_2 and $\mathbf{v}_2$.

Step 5: Repeat: $\mathbf{A}_3 = \mathbf{A}_2 - \lambda_2 \hat{\mathbf{v}}_2 \hat{\mathbf{v}}_2^T \rightarrow$ power method $\rightarrow \lambda_3, \dots$

Caveat: Each deflation uses the *computed* eigenvector, which has small errors. Those errors propagate into $\mathbf{A}_2$, then compound in $\mathbf{A}_3$, etc.

So this works well for the *first few* eigenvalues, but accuracy degrades if you try to peel off too many.

Thank you!